

Minimizing Loss Functions in Deep Learning

1 Setup: What Are We Minimizing?

We train a model with parameters (weights) $\mathbf{w}$ by minimizing a loss. For a single example $(\mathbf{x}, y)$, write $\ell(\mathbf{w}; \mathbf{x}, y)$. For n training examples,

$$L(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{w}; \mathbf{x}_i, y_i).$$

The goal is to find $\mathbf{w}^*$ that (approximately) minimizes $L(\mathbf{w})$.

2 Gradient Descent (GD)

Idea. Move parameters in the direction that most decreases the loss: the negative gradient.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t),$$

where $\eta > 0$ is the *learning rate*.

Why this helps. A first-order Taylor approximation says $L(\mathbf{w} - \delta) \approx L(\mathbf{w}) - \nabla L(\mathbf{w})^\top \delta$; choosing $\delta = \eta \nabla L(\mathbf{w})$ decreases L when η is small enough.

Choosing η . Too small $\Rightarrow$ slow; too large $\Rightarrow$ divergence/oscillation. Start with 10^{-3} to 10^{-1} and tune.

Toy example: linear regression (MSE). Let $A \in \mathbb{R}^{n \times d}$ be the data matrix (rows are $\mathbf{x}_i^\top$) and $\mathbf{y} \in \mathbb{R}^n$ targets. With

$$L(\mathbf{w}) = \frac{1}{2n} \|A\mathbf{w} - \mathbf{y}\|_2^2, \quad \nabla L(\mathbf{w}) = \frac{1}{n} A^\top (A\mathbf{w} - \mathbf{y}),$$

the update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{1}{n} A^\top (A\mathbf{w}_t - \mathbf{y}).$$

Full-batch GD pseudocode.

```
w = initialize()
for t = 0..T-1:
    g = gradient_of_L_on_full_data(w)
    w = w - eta * g
```

3 Stochastic & Mini-Batch Gradient Descent (SGD)

Key idea. Use a small random subset (mini-batch) to *estimate* the gradient—much cheaper per step.

Let $B_t \subset \{1, \dots, n\}$ with $|B_t| = m$ (e.g., 32, 64, 128).

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{1}{m} \sum_{i \in B_t} \nabla \ell(\mathbf{w}_t; \mathbf{x}_i, y_i).$$

This is a noisy but (typically) unbiased estimate of $\nabla L(\mathbf{w}_t)$.

Batch size. Small m : faster, noisier updates (often good for generalization). Large m : smoother but costlier steps. Typical $m \in [32, 512]$, hardware-dependent.

Mini-batch loop.

```
w = initialize()
for epoch in 1..E:
    shuffle data
    for each mini-batch B:
        g = (1/|B|) * sum_{i in B} grad l(w; x_i, y_i)
        w = w - eta * g
```

4 Momentum and Nesterov Momentum

Problem. Plain SGD can zig-zag in narrow valleys and be slowed by noise.

Momentum. Keep a velocity $\mathbf{v}$ that accumulates gradients:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \mathbf{g}_t, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{v}_{t+1},$$

with $\mathbf{g}_t$ the (mini-batch) gradient and $\beta \in [0, 1)$ (e.g., 0.9).

Nesterov (look-ahead).

$$\tilde{\mathbf{w}}_t = \mathbf{w}_t - \eta \beta \mathbf{v}_t, \quad \mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \nabla L(\tilde{\mathbf{w}}_t), \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{v}_{t+1}.$$

Intuition: like pushing a ball downhill; it builds speed along consistent directions and damps oscillations.

5 Adaptive Learning-Rate Methods

Different parameters can benefit from different step sizes.

AdaGrad

Accumulate *squared* gradients per coordinate:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \mathbf{g}_t \odot \mathbf{g}_t, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_{t+1}} + \epsilon}.$$

Good for sparse features; steps shrink over time.

RMSProp

Use an exponential moving average (EMA) of squared gradients:

$$\mathbf{s}_{t+1} = \rho \mathbf{s}_t + (1 - \rho) \mathbf{g}_t \odot \mathbf{g}_t, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_{t+1} + \epsilon}}.$$

Typical $\rho \approx 0.9$.

Adam

Adam combines momentum and RMSProp with bias correction:

$$\begin{aligned} \mathbf{m}_{t+1} &= \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{v}_{t+1} &= \beta_2 \mathbf{v}_t + (1 - \beta_2) (\mathbf{g}_t \odot \mathbf{g}_t), \\ \hat{\mathbf{m}}_{t+1} &= \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}, \\ \mathbf{w}_{t+1} &= \mathbf{w}_t - \eta \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon}}. \end{aligned}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. *Rule of thumb:* Adam/RMSProp start fast; SGD+momentum sometimes yields stronger final generalization.

6 Learning-Rate Schedules & Useful Tricks

- **Step decay:** multiply η by $\gamma < 1$ every k epochs.
- **Cosine decay:** smoothly decrease η over time.
- **Warmup:** start with a small η , ramp up for a few steps/epochs.
- **Early stopping:** stop when validation loss stops improving.
- **Weight decay (L2):** add $\lambda \|\mathbf{w}\|_2^2$. With SGD: $\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w} - \eta \mathbf{g}$. With Adam, prefer decoupled decay (AdamW).
- **Gradient clipping:** cap the norm of the gradient to avoid exploding updates.

7 Common Pitfalls (and Quick Fixes)

- **Diverging loss / NaNs:** lower η ; normalize inputs; clip gradients.
- **Loss plateaus:** increase η slightly; try momentum/Adam; check for vanishing gradients; use BatchNorm/LayerNorm.
- **Overfitting:** use weight decay, dropout, data augmentation, or early stopping.
- **Very large batches:** stable training but worse validation; try smaller batches or more regularization.

8 A Minimal Training Loop (Template)

```
initialize w
choose optimizer (SGD, SGD+momentum, Adam, ...)
for epoch = 1..E:
    shuffle data
    for each mini-batch B:
        g = (1/|B|) * sum_{i in B} grad l(w; x_i, y_i)
        w = optimizer_update(w, g) # includes momentum/Adam, weight decay, etc.
    evaluate train/val loss (and accuracy for classification)
    Adjust the learning rate if using a schedule
```

9 Quick Practice

1. **MSE (one sample).** For $\ell(\mathbf{w}) = \frac{1}{2}(\mathbf{w}^\top \mathbf{x} - y)^2$, compute $\nabla \ell$.
Answer: $(\mathbf{w}^\top \mathbf{x} - y) \mathbf{x}$.
2. **Logistic regression (one sample).** $\ell(\mathbf{w}) = \log(1 + e^{-y \mathbf{w}^\top \mathbf{x}})$. Compute $\nabla \ell$.
Answer: $-y \sigma(-y \mathbf{w}^\top \mathbf{x}) \mathbf{x}$.
3. **Momentum intuition.** In a narrow valley, why can momentum converge faster than plain SGD?
Hint: momentum damps oscillations across the valley while accumulating speed along the valley floor.

One-Page Cheat Sheet

- **GD:** $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$.
- **SGD (mini-batch):** $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{m} \sum_{i \in B} \nabla \ell_i$.
- **Momentum:** $\mathbf{v} \leftarrow \beta \mathbf{v} + (1 - \beta) \mathbf{g}$, $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{v}$.
- **Nesterov:** look ahead before computing the gradient.
- **RMSProp:** scale by $1/\sqrt{\text{EMA of } \mathbf{g}^2}$.
- **Adam:** momentum + RMSProp + bias correction.
- **Weight decay (decoupled/AdamW):** $\mathbf{w} \leftarrow (1 - \eta \lambda) \mathbf{w} - \eta \text{update}$.